MASTER OF COMPUTER APPLICATIONS

PRACTICAL RECORD WORK

ON

20MCA132-OBJECT ORIENTED PROGRAMMING LAB

SUBMITTED BY

NAME: MUHAMMED NAJI A C

REG.NO: VDA25MCA-2027

DEPARTMENT OF COMPUTERAPPLICATIONS

COLLEGE OF ENGINEERING VADAKARA

(CAPE -GOVT. OF KERALA)

APRIL 2026

DEPARTMENT OF COMPUTER APPLICATIONS
COLLEGE OF ENGINEERING VADAKARA
(CAPE -GOVT. OF KERALA)

CERTIFICATE

Certified that this is the Bonafide record work on the practical course **20MCA132- OBJECT ORIENTED PROGRAMMING LAB** done and prepared by Mr.**MUHAMMED NAJI AC (Reg.No VDA25MCA- 2027**), 2nd Semester MCA (2025-27 Batch) student of Department of Computer Applications at College of Engineering Vadakara, in the partial fulfilment of the award of MCA Degree of APJ Abdul Kalam Technological University (KTU).

Faculty-in-Charge Head of the Department

(Office Seal)

Internal Examiners: External Examiners:

INDEX

SL. NO	EXPERIMENT	PAGE. NO	DATE	REMARKS
1	Define a class ‘product ‘with data members pcode, pname and price. Create 3 objects of 1 the class and find the product having the lowest price.	1	06-02-2026	
2	Read 2 matrices from the console and perform matrix addition.	3	06-02-2026	
3	Add complex numbers.	6	06-02-2026	
4	Read a matrix from the console and check whether it is symmetric or not.	8	13-02-2026	
5	Create CPU with attribute price. Create inner class Processor (no. of cores, manufacturer) and static nested class RAM (memory, manufacturer). Create an object of CPU and print information of Processor and RAM.	11	13-02-2026	
6	Program to Sort strings	14	19-02-2026	
7	Search an element in an array	16	19-02-2026	
8	Perform string manipulations.	19	25-02-2026	
9	Program to create class for Employee having attributes eNo, eName eSalary. Read n employ information and Search for an employee given eNo, using the concept of Array of Objects.	21	25-02-2026	
10	Area of different shapes using overloaded functions.	24	04-03-2026	
11	Program to create a class Employee and a subclass Teacher with additional details. Use an array of objects to display details of N teachers.	28	04-03-2026	
12	Program to implement multilevel inheritance using classes Person, Employee, and Teacher with an array of objects to display details of N teachers.	32	15-03-2026	

13	Write a program has class Publisher, Book, Literature and Fiction. Read the information 2and print the details of books from either the category, using inheritance.	35	20-03-2026	
14	Create classes Student and Sports. Create another class Result inherited from Student and Sports. Display the academic and sports score of a student.	39	20-03-2026	
15	Create an interface having prototypes of functions area () and perimeter (). Create two classes Circle and Rectangle which implements the above interface. Create a menu driven program to find area and perimeter of objects.	42	25-03-2026	
16	Prepare bill with the given format using calculate method from interface. Order No.	45	25-03-2026	
17	Create a Graphics package that has classes and interfaces for figures Rectangle, Triangle, Square and Circle. Test the package by finding the area of these figures.	48	27-03-2026	
18	Create an Arithmetic package that has classes and interfaces for the 4 basic arithmetic operations. Test the package by implementing all operations on two given numbers	50	27-03-2026	

PROGRAM 1:

AIM:

Define a class ‘product ‘with data members pcode, pname and price. Create 3 objects of the class and find the product having the lowest price.

ALGORITHM:

Step 1: Start

Step 2: Define a class Product with:
  a) Data members: pcode, pname, price
  b) Constructor to initialize the data members

Step 3: In the main method:
  a) Create object p1 with values:
     pcode = 1, pname = "Laptop", price = 43000
  b) Create object p2 with values:
     pcode = 2, pname = "TV", price = 20000
  c) Create object p3 with values:
     pcode = 3, pname = "Mobile", price = 18000

Step 4: Compare the prices of the three products:
  a) If (p1.price < p2.price) AND (p1.price < p3.price) then
     Print "Product p1 has the lowest price"

  b) Else if (p2.price < p1.price) AND (p2.price < p3.price) then
     Print "Product p2 has the lowest price"

  c) Else
     Print "Product p3 has the lowest price"

Step 5: End

SOURCE CODE:

```
class Product{

String name;

double price;

Product(String n,double p){

name=n;

price=p;
```

```
}
public static void main(String[] args){
Product p1=new Product("pen",10);
Product p2=new Product("book",50);
Product p3=new Product("pencil",5);
Product lowest=p1;
if (p2.price<lowest.price){lowest=p2;}
if (p3.price<lowest.price){lowest=p3;}
System.out.println("Lowest price product :");
System.out.println("Name :"+lowest.name);
System.out.println("Price :"+lowest.price);
}
}
```

OUTPUT:

RESULT:

Successfully executed

PROGRAM 2:

AIM:

Read 2 matrices from the console and perform matrix addition.

ALGORITHM:

Step 1: Start

Step 2: Define a class AddMatrix with:
- a) Integer variables r, c for number of rows and columns
- b) A 2D integer array arr[10][10] to store matrix elements
- c) A method get() to input matrix dimensions and elements
- d) A static method sum(AddMatrix c1, AddMatrix c2) to add two matrices
- e) A method display() to print the matrix

Step 3: Define method get()
- a) Input number of rows r and columns c
- b) For i = 0 to r-1
  - For j = 0 to c-1
    - Read arr[i][j]

Step 4: Define static method sum(AddMatrix c1, AddMatrix c2)
- a) Create object temp of class AddMatrix
- b) If (c1.r == c2.r) AND (c1.c == c2.c) then
  - i) Set temp.r = c1.r and temp.c = c1.c
  - ii) For i = 0 to temp.r - 1
    - For j = 0 to temp.c - 1
      - temp.arr[i][j] = c1.arr[i][j] + c2.arr[i][j]
- c) Else
  - Print "Addition not possible"
- d) Return temp

Step 5: Define method display()
- a) For i = 0 to r-1
  - For j = 0 to c-1
    - Print arr[i][j]

Step 6: In the main() method
- a) Create objects m1, m2, and result of class AddMatrix
- b) Call m1.get() and m2.get()
- c) Set result = sum(m1, m2)
- d) Call result.display() to print the result

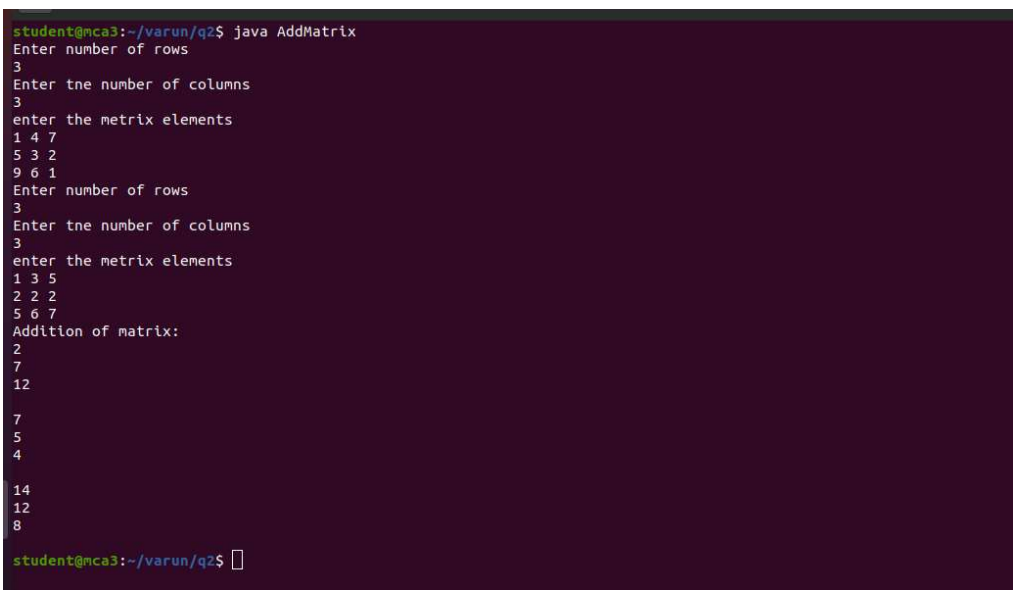
Step 7: End

SOURCE CODE:

```
import java.util.Scanner;
class AddMatrix{
int r,c;
int arr[][] = new int[10][10];
public void get(){

Scanner sc=new Scanner(System.in);
System.out.println("Enter number of rows");
r=sc.nextInt();
System.out.println("Enter the number of columns");
c=sc.nextInt();
System.out.println("enter the matrix elements");
for(int i=0;i<r;i++){
for(int j=0;j<c;j++){
arr[i][j]=sc.nextInt();
}
}
}
public void display(){
for(int i=0;i<r;i++){
for(int j=0;j<c;j++){
System.out.println(arr[i][j]+" ");
}
System.out.println();
}
}
public static AddMatrix sum(AddMatrix m1,AddMatrix m2){
AddMatrix temp= new AddMatrix();
if(m1.r==m2.r && m1.c==m2.c){
temp.r=m1.r;
temp.c=m1.c;
```

```
for(int i=0;i<m1.r;i++){
for(int j=0;j<m1.c;j++){
temp.arr[i][j]=m1.arr[i][j]+m2.arr[i][j];}
}
else{
System.out.println("Addition not possible");
}
return temp;
}
public static void main (String[] args){
AddMatrix m1= new AddMatrix();
AddMatrix m2= new AddMatrix();
m1.get();
m2.get();
AddMatrix result= sum(m1,m2);
System.out.println("Addition of matrix:");
result.display();}
}
```

OUTPUT:



```
student@mca3:~/varun/q2$ java AddMatrix
Enter number of rows
3
Enter tne number of columns
3
enter the metrix elements
1 4 7
5 3 2
9 6 1
Enter number of rows
3
Enter tne number of columns
3
enter the metrix elements
1 3 5
2 2 2
5 6 7
Addition of matrix:
2
7
12

7
5
4

14
12
8

student@mca3:~/varun/q2$
```

RESULT:

Successfully executed

PROGRAM 3:

AIM:

Add 2 complex numbers

ALGORITHM:

Step 1: Start
Step 2: Define a class Complex with:
  a) Data members:
    i) real → to store real part of complex number
    ii) imaginary → to store imaginary part of complex number
  b) A constructor to initialize the data members with user-defined values
Step 3: Define a static method sum(Complex c1, Complex c2)
  a) Create a new Complex object temp initialized with (0, 0)
  b) Set temp.real = c1.real + c2.real
  c) Set temp.imaginary = c1.imaginary + c2.imaginary
  d) Return temp
Step 4: In the main() method:
  a) Create object first with values (1, 2)
  b) Create object second with values (32, 22)
  c) Call sum(first, second) and store result in object temp
  d) Print the result in the format: real + imaginary i
Step 5: End

SOURCE CODE:

```
public class Complex {
int real;
int imaginary;
public Complex(int real, int imaginary) {
this.real = real;
this.imaginary = imaginary;
}
public static Complex sum(Complex c1, Complex c2) {
Complex temp = new Complex(0, 0);
temp.real = c1.real + c2.real;
```

```
temp.imaginary = c1.imaginary + c2.imaginary;
return temp;
}
public static void main(String[] args) {
Complex first = new Complex(1, 2);
Complex second = new Complex(32, 22);
Complex temp = sum(first, second);
System.out.println("Sum: " + temp.real + " + " + temp.imaginary + "i");
}
}
```

OUTPUT:

```
student@mca3:~/varun/q3$ java Complex
Sum: 33 + 24i
student@mca3:~/varun/q3$
```

RESULT:

Successfully executed

PROGRAM 4:

AIM:

Read a matrix from the console and check whether it is symmetric or not.

ALGORITHM:

Step 1: Start
Step 2: Define a class Symmetric with:
  a) Integer variables r and c for number of rows and columns
  b) A 2D integer array arr to store matrix elements
  c) A method get() to input matrix elements
  d) A method isSym() to check for symmetry
Step 3: Define method get()
  a) Prompt the user to input the number of rows r and columns c
  b) Create a matrix of size r x c
  c) Use nested loops to input the elements of the matrix from the user
Step 4: Define method isSym()
  a) Initialize a flag variable to 0
  b) Use nested loops to compare element arr[i][j] with arr[j][i] for all i and j
  c) If any mismatch is found:
    Set flag = 1
    Break out of the loop
  d) After checking:
    If flag == 0, print "this is symmetric"
    Else, print "this is not symmetric"
Step 5: In the main() method:
  a) Create an object matrix of class Symmetric
  b) Call matrix.get() to input matrix
  c) Call matrix.isSym() to check symmetry and display result
Step 6: End

SOURCE CODE:

```
import java.util.Scanner;

public class Symmetric {

int r, c;

int[][] arr;

public void get() {

Scanner sc = new Scanner(System.in);
```

```
System.out.println("Enter number of rows:");
r = sc.nextInt();
System.out.println("Enter number of columns:");
c = sc.nextInt();
arr = new int[r][c];
System.out.println("Enter matrix elements:");
for (int i = 0; i < r; i++) {
for (int j = 0; j < c; j++) {
arr[i][j] = sc.nextInt();
}
}
}
 public void isSym() {
int flag = 0;
if (r != c) {
System.out.println("Matrix is not symmetric");
return;
}
for (int i = 0; i < r; i++) {
for (int j = 0; j < c; j++) {
if (arr[i][j] != arr[j][i]) {
flag = 1;
break;
}
}
}
if (flag == 0)
System.out.println("Matrix is symmetric");
else
```

```
System.out.println("Matrix is not symmetric");
}
public static void main(String[] args) {
Symmetric matrix = new Symmetric();
matrix.get();
matrix.isSym();
}
}
```

OUTPUT:

```
student@mca3:~/varun/q4$ java Symmetric
Enter number of rows:
3
Enter number of columns:
3
Enter matrix elements:
1 4 8
5 3 9
3 8 1
Matrix is not symmetric
student@mca3:~/varun/q4$ java Symmetric
Enter number of rows:
3
Enter number of columns:
3
Enter matrix elements:
4 5 6
5 8 9
6 9 7
Matrix is symmetric
student@mca3:~/varun/q4$
```

RESULT:

Successfully executed

PROGRAM 5:

AIM:

Create CPU with attribute price. Create inner class Processor (no. of cores, manufacturer) and static nested class RAM (memory, manufacturer). Create an object of CPU and print information of Processor and RAM.

ALGORITHM:

Step 1: Start
Step 2: Define a base class Cpu
  2.1. Declare data member price
  2.2. Define constructor to initialize price
  2.3. Define method display() to show CPU price
Step 3: Define a derived class Processor extending Cpu
  3.1. Declare data members cores and manufacturer
  3.2. Define constructor with parameters for price, cores, and manufacturer
  3.3. Use super(price) to call the base class constructor
  3.4. Initialize cores and manufacturer
  3.5. Define method displayProcessor() to:
    3.5.1. Call display() to print CPU price
    3.5.2. Print number of cores
    3.5.3. Print processor manufacturer
Step 4: Define another derived class Ram extending Cpu
  4.1. Declare data members memory and manufacturer
  4.2. Define constructor with parameters for price, memory, and manufacturer
  4.3. Use super(price) to call base class constructor
  4.4. Initialize memory and manufacturer
  4.5. Define method displayRam() to:
    4.5.1. Print memory size
    4.5.2. Print RAM manufacturer
Step 5: In the main() method:
  5.1. Create an object processor of class Processor with values (23000, 8, "Intel")
  5.2. Create an object ram of class Ram with values (23000, 16, "Corsair")
  5.3. Call processor.displayProcessor()
  5.4. Call ram.displayRam()
Step 6: End

SOURCE CODE:

```
class Cpu
{
int price;
```

```
public Cpu(int p)
{
this.price=p;
}
void display()
{
System.out.println("price of cpu :"+this.price);
}
class Processor
{
int cores;
String manufacture;
Processor(int core,String manf)
{
this.cores=core;
this.manufacture=manf;
}
void display()
{
System.out.println("no of cores:"+this.cores);
System.out.println("processor manufacture:"+this.manufacture);
}
}
static class Ram
{
int memory;
String manufacture;
Ram(int n,String m)
{
this.memory=n;
this.manufacture=m;
}
void display()
```

```
{
System.out.println("memory size:"+this.memory);
System.out.println("memory manufacture:"+this.manufacture);
}
}
public static void main(String [] args)
{
Cpu intel=new Cpu(23000);
Cpu.Processor p=intel.new Processor(8,"intel");
Cpu.Ram ram=new Ram(64,"asus");
intel.display();
p.display();
ram.display();
}
}
```

OUTPUT:

```
student@mca3:~/varun/q5$ javac Cpu.java
student@mca3:~/varun/q5$ java Cpu
price of cpu :23000
no of cores:8
processor manufacture:intel
memory size:64
memory manufacture:asus
student@mca3:~/varun/q5$
```

RESULT:

Successfully executed

PROGRAM 6:

AIM:

Program to Sort strings

ALGORITHM:

Step 1: Start
Step 2: Create a Scanner object to read input from the user
Step 3: Prompt the user to enter the number of strings (n)
Step 4: Read the integer n from the user
Step 5: Create a string array of size n to store the input strings
Step 6: Prompt the user to enter n strings
Step 7: Use a loop to read n strings and store them in the array:
  a) For i from 0 to n-1
    Read string and store in strings[i]
Step 8: Sort the string array using Arrays.sort(), which arranges strings in lexicographical order (dictionary order)
Step 9: Display the sorted list of strings using a loop:
  a) For each string in the sorted array, print the string
Step 10: Close the Scanner object
Step 11: End

SOURCE CODE:

```
import java.util.Arrays;
import java.util.Scanner;
public class StringSort
{
public static void main(String[] args)
{
Scanner scanner=new Scanner(System.in);
System.out.println("Enter the number of strings:");
int n=scanner.nextInt();
String[] strings=new String[n];
System.out.println("Enter the strings:");
for(int i=0;i<n;i++)
{
strings[i]=scanner.next();
```

```
}
Arrays.sort(strings);
System.out.println("\nSorted strings:");
for(int i=0;i<n;i++)
{
System.out.println(strings[i]);
}
scanner.close();
}
}
```

OUTPUT:

```
student@cev:~/alfa$ javac Sort.java
student@cev:~/alfa$ java Sort
Enter the number of strings:
3
Enter the strings:
rahul
anil
devu

Sorted strings:
anil
devu
rahul
student@cev:~/alfa$
```

RESULT:

Successfully executed

PROGRAM 7:

AIM:

Search an element in an array

ALGORITHM:

Step 1: Start
Step 2: Declare an array arr[10] and a variable count to store the number of elements
Step 3: Define method get() to input array elements:
3.1. Create a Scanner object
3.2. Prompt the user to enter the number of elements (count)
3.3. Read the count
3.4. Prompt the user to enter array elements
3.5. Use a loop to read each element and store in arr[i] for i = 0 to count - 1
Step 4: Define method itemSearch() to search for an element:
4.1. Prompt the user to enter the search element n
4.2. Initialize flag = 0
4.3. Use a loop from i = 0 to count - 1:
If arr[i] == n:
Set flag = 1
Break the loop
4.4. If flag == 1:
Print that the element is found at position i + 1
4.5. Else:
Print **"element not found"**
Step 5: In the main() method:
5.1. Create an object s1 of class search
5.2. Call s1.get() to input array elements
5.3. Call s1.itemSearch() to perform the search
Step 6: End

SOURCE CODE:

```
import java.util.*;
public class Search
{
int[] arr=new int[10];
int count;

public void get()
```

```
{
Scanner sc=new Scanner(System.in);
System.out.println("enter the count of array:");
this.count=sc.nextInt();
System.out.println("enter the array elements:");
for(int i=0;i<this.count;i++)
{
this.arr[i]=sc.nextInt();
}
}

public void itemSearch()
{
int n,flag=0,pos=0;
Scanner sc=new Scanner(System.in);
System.out.println("enter the element you want to search:");
n=sc.nextInt();
for(int i=0;i<this.count;i++)
{
if(this.arr[i]==n)
{
flag=1;
pos=i;
break;
}
}
if(flag==1)
{
System.out.println("element is found at position "+(pos+1));
}
else
{
System.out.println("element is not found");
```

```
}
}

public static void main(String[] args)
{
Search s1=new Search();
s1.get();
s1.itemSearch();
}
}
```

OUTPUT:

```
student@mca3:~/varun/q6$ java Search
enter the count of array:
3
enter the array elements:
7
8
10
enter the element you want to search:
3
element is not found
student@mca3:~/varun/q6$ java Search
enter the count of array:
4
enter the array elements:
1
2
9
8
enter the element you want to search:
2
element is found at position 2
student@mca3:~/varun/q6$
```

RESULT:

Successfully executed

PROGRAM 8:

AIM:

Perform string manipulations.

ALGORITHM:

Step 1: Start
Step 2: Initialize a string variable str with the value "Hello, World!"
Step 3: Find the length of the string using length() method
- Store the result in length
- Print the length

Step 4: Convert the string to uppercase using toUpperCase()
- Store in uppercase
- Print the result

Step 5: Convert the string to lowercase using toLowerCase()
- Store in lowercase
- Print the result

Step 6: Replace all occurrences of the character 'o' with 'x' using replace()
- Store in replaced
- Print the result

Step 7: Extract a substring from index 7 to 11 (end index is exclusive) using substring(7, 12)
- Store in substring
- Print the result

Step 8: Check if the string starts with "Hello" using startsWith()
- Store result in startsWith
- Print the result

Step 9: Check if the string ends with "World!" using endsWith()
- Store result in endsWith
- Print the result

Step 10: Trim leading and trailing spaces from the string " Hello, World! " using trim()
- Store in trimmed
- Print the result

Step 11: End

SOURCE CODE:

```
public class StringManipulation {
    public static void main(String[] args) {
        String original = "Hello, World!";
        String reversed = new StringBuilder(original).reverse().toString();
```

```
        System.out.println("Reversed: " + reversed);
        String upperCase = original.toUpperCase();
        System.out.println("Uppercase: " + upperCase);
        String lowerCase = original.toLowerCase();
        System.out.println("Lowercase: " + lowerCase);
        boolean containsSubstring = original.contains("World");
        System.out.println("Contains 'World': " + containsSubstring);
        String replaced = original.replace("World", "Java");
        System.out.println("Replaced 'World' with 'Java': " + replaced);
        String substring = original.substring(7, 12);
        System.out.println("Substring (7,12): " + substring);
        String trimmed = "   Hello Java!   ".trim();
        System.out.println("Trimmed: '" + trimmed + "'");
        String[] words = original.split(" ");
        System.out.println("Words in the original string:");
        for (String word : words) {
            System.out.println(word);
        }
    }
}
```

OUTPUT:

```
student@mca3:~/varun/q8$ java StringManipulation
Reversed: !dlroW ,olleH
Uppercase: HELLO, WORLD!
Lowercase: hello, world!
Contains 'World': true
Replaced 'World' with 'Java': Hello, Java!
Substring (7,12): World
Trimmed: 'Hello Java!'
Words in the original string:
Hello,
World!
student@mca3:~/varun/q8$
```

RESULT:

Successfully executed

PROGRAM 9:

AIM:

Program to create class for Employee having attributes eNo, eName eSalary. Read n employ information and Search for an employee given eNo, using the concept of Array of Objects.

ALGORITHM:

Step 1: Start
Step 2: Define a class Employee with data members:
- eno (Employee ID)
- ename (Employee Name)
- salary (Employee Salary)

Step 3: Define the read() method to:
- Prompt user to input eno, ename, and salary
- Read and store these values

Step 4: Define the display() method to:
- Print eno, ename, and salary

Step 5: In the main() method:
- Set n = 3 (number of employees)
- Create an array emp[] of type Employee with n elements

Step 6: For i = 0 to n-1:
- Initialize emp[i] as a new Employee object
- Call emp[i].read() to read details

Step 7: Print "Search"
Step 8: Repeat infinitely (or until externally stopped):
- Prompt user to enter an Employee ID
- Read the input as no
- For i = 0 to n-1:
  - If emp[i].eno equals no:
    - Call emp[i].display() to show details
    - Break the loop

Step 9: End

SOURCE CODE:

```
import java.util.*;

class Employee

{

int eno;

String ename;

int salary;
```

```
public void read(){
Scanner sc=new Scanner(System.in);
System.out.println("enter the id:");
this.eno=sc.nextInt();
sc.nextLine();
System.out.println("enter the employee name:");
this.ename=sc.nextLine();
System.out.println("enter the salary:");
this.salary=sc.nextInt();
}
public void display(){
System.out.println("employee id:"+this.eno);
System.out.println("employee name:"+this.ename);
System.out.println("employee salary:"+this.salary);
}
public static void main(String[]args){
Scanner sc=new Scanner(System.in);
int i,n;
System.out.println("enter no of employees");
n=sc.nextInt();
Employee emp[]=new Employee[n];
for(i=0;i<n;i++){
emp[i]=new Employee();
emp[i].read();
}
System.out.println("Search");
while(true){
System.out.print("Enter id to search(or -1 to exit):");
int no=sc.nextInt();
if(no==-1) break;
boolean found=false;
for(i=0;i<n;i++){
if(emp[i].eno==no){
```

```
emp[i].display();
found=true;
break;
}
}
if(!found){
System.out.println("Employee not found");
}
}
}
}
```

OUTPUT:

```
student@mca3:~/varun/q9$ gedit Employee.java
student@mca3:~/varun/q9$ javac Employee.java
student@mca3:~/varun/q9$ java Employee
enter no of employees
2
enter the id:
100
enter the employee name:
meenakshi
enter the salary:
100000000
enter the id:
101
enter the employee name:
varun tp
enter the salary:
300
Search
Enter id to search(or -1 to exit):2
Employee not found
Enter id to search(or -1 to exit):101
employee id:101
employee name:varun tp
employee salary:300
Enter id to search(or -1 to exit):100
employee id:100
employee name:meenakshi
employee salary:100000000
Enter id to search(or -1 to exit):-1
student@mca3:~/varun/q9$
```

RESULT:

Successfully executed

PROGRAM 10:

AIM:

Area of different shapes using overloaded functions.

ALGORITHM:

Step 1: Start
Step 2: Define a class Area
Step 3: Inside the class, define overloaded methods named area:
  3.1 For Circle:
    - Input: r (radius, double)
    - Compute area as π * r * r
    - Return result
  3.2 For Rectangle:
    - Input: l (length, double), w (width, double)
    - Compute area as l * w
    - Return result
  3.3 For Triangle:
    - Input: b (base, int), h (height, int)
    - Compute area as -0.5 * b * h
    - Return result
  3.4 For Square:
    - Input: l (side, int)
    - Compute area as l * l
    - Return result
Step 4: In the main() method:
  - Create an object obj of class Area
  - Call obj.area(5.0) → prints area of circle
  - Call obj.area(3.0, 4.0) → prints area of rectangle
  - Call obj.area(4, 3) → prints area of triangle
  - Call obj.area(5) → prints area of square
Step 5: End

SOURCE CODE:

```
import java.util.*;
class Employee
{
int eno;
String ename;
```

```
int salary;
public void read(){
Scanner sc=new Scanner(System.in);
System.out.println("enter the id:");
this.eno=sc.nextInt();
sc.nextLine();
System.out.println("enter the employee name:");
this.ename=sc.nextLine();
System.out.println("enter the salary:");
this.salary=sc.nextInt();
}
public void display(){
System.out.println("employee id:"+this.eno);
System.out.println("employee name:"+this.ename);
System.out.println("employee salary:"+this.salary);
}
public static void main(String[]args){
Scanner sc=new Scanner(System.in);
int i,n;
System.out.println("enter no of employees");
n=sc.nextInt();
Employee emp[]=new Employee[n];
for(i=0;i<n;i++){
emp[i]=new Employee();
emp[i].read();
}
System.out.println("Search");
while(true){
System.out.print("Enter id to search(or -1 to exit):");
int no=sc.nextInt();
if(no==-1) break;
boolean found=false;
for(i=0;i<n;i++){
```

```
if(emp[i].eno==no){
emp[i].display();
found=true;
break;
}
}
if(!found){
System.out.println("Employee not found");
}
}
}
}public class Area
{
public double area(double r)
{
return Math.PI*r*r;
}

public double area(double l,double w)
{
return l*w;
}

public double area(int b,int h)
{
return 0.5*b*h;
}

public int area(int l)
{
return l*l;
}
```

```
public static void main(String []args)
{
Area obj=new Area();
System.out.println("area of circle r=5:"+obj.area(5.0));
System.out.println("area of rectangle(3,4):"+obj.area(3.0,4.0));
System.out.println("area of triangle(4,3):"+obj.area(4,3));
System.out.println("area of square(5):"+obj.area(5));
}
}
```

OUTPUT:

```
student@mca3:~/varun/q10$ javac Area.java
student@mca3:~/varun/q10$ java Area
area of circle r=5:78.53981633974483
area of rectangle(3,4):12.0
area of triangle(4,3):6.0
area of square(5):25
student@mca3:~/varun/q10$
```

RESULT:

Successfully executed

PROGRAM 11:

AIM:

Create a class 'Employee' with data members Empid, Name, Salary, Address and constructors to initialize the data members. Create another class 'Teacher' that inherit the properties of class employee and contain its own data members department, Subjects taught and constructors to initialize these data members and also include display function to display all the data members. Use array of objects to display details of N teachers

ALGORITHM:

Step 1: Start
Step 2: Define the class Employee with:
- Data members: id, name, sal, address
- Constructor to initialize data members
- Method display() to print employee details

Step 3: Define the class teacher that extends Employee:
- Additional data members: dept, sub
- Constructor to initialize both base and derived class data
- Override display() to include teacher-specific data

Step 4: In the main() method (inside class co3_2):
- Create a Scanner object to read input
- Prompt the user to enter number n (number of teachers)
- Create an array t of teacher objects of size n

Step 5: Repeat for each teacher from i = 0 to n - 1:
- Prompt and read values: id, name, salary, address, department, subject
- Create a new teacher object using these values and store in array t[i]

Step 6: Display the details of all teachers:
- Loop through the array and call display() on each t[i]

Step 7: End

SOURCE CODE:

```
import java.util.Scanner;

class employee

{

int id;

String name;

int sal;

String address;

employee(int id,String name,int sal,String address)
```

```
{
this.id=id;
this.name=name;
this.sal=sal;
this.address=address;
}
public void display()
{
System.out.println("Employee id:"+this.id);
System.out.println("Employee name:"+this.name);
System.out.println("Employee salary:"+this.sal);
System.out.println("Employee address:"+this.address);
}
}
class teacher extends employee
{
String dept;
String sub;
teacher(int id,String name,int sal,String address,String dept,String sub)
{
super(id,name,sal,address);
this.dept=dept;
this.sub=sub;
}
public void display()
{
super.display();
System.out.println("Department:"+this.dept);
System.out.println("Subject:"+this.sub);
}
}
public class co3_2
{
```

```
public static void main(String[]args)
{
Scanner sc=new Scanner(System.in);
System.out.println("Enter the number of teachers:");
int n=sc.nextInt();
sc.nextLine();
teacher[]t=new teacher[n];
for(int i=0;i<n;i++)
{
System.out.println("Enter details of teacher "+(i+1)+":");
System.out.println("ID:");
int id=sc.nextInt();
sc.nextLine();
System.out.println("Name:");
String name=sc.nextLine();
System.out.println("Salary:");
int sal=sc.nextInt();
sc.nextLine();
System.out.println("Address:");
String address=sc.nextLine();
System.out.println("Department:");
String dept=sc.nextLine();
System.out.println("Subject:");
String sub=sc.nextLine();
t[i]=new teacher(id,name,sal,address,dept,sub);
}
System.out.println("\nTeacher details:\n");
for(int i=0;i<n;i++)
{
t[i].display();
System.out.println();
}
sc.close();}}
```

OUTPUT:

```
student@mca3:~/varun/q11$ java co3_2
Enter the number of teachers:
2
Enter details of teacher 1:
ID:
100
Name:
razi
Salary:
1000
Address:
mahe
Department:
mca
Subject:
vc
Enter details of teacher 2:
ID:
101
Name:
sreehari
Salary:
1000
Address:
vadakara
Department:
mca
Subject:
ipr

Teacher details:

Employee id:100
Employee name:razi
Employee salary:1000
Employee address:mahe
Department:mca
Subject:vc
```

```
1000
Address:
mahe
Department:
mca
Subject:
vc
Enter details of teacher 2:
ID:
101
Name:
sreehari
Salary:
1000
Address:
vadakara
Department:
mca
Subject:
ipr

Teacher details:

Employee id:100
Employee name:razi
Employee salary:1000
Employee address:mahe
Department:mca
Subject:vc

Employee id:101
Employee name:sreehari
Employee salary:1000
Employee address:vadakara
Department:mca
Subject:ipr
```

RESULT:

Successfully executed

PROGRAM 12:

AIM:

Create a class 'Person' with data members Name, Gender, Address, Age and a constructor to initialize the data members and another class 'Employee' that inherits the properties of class Person and also contains its own data members like Empid, Company_name, Qualification, Salary and its own constructor. Create another class 'Teacher' that inherits the properties of class Employee and contains its own data members like Subject, Department, Teacherid and also contain constructors and methods to display the data members. Use array of objects to display details of N teachers.

ALGORITHM:

Step 1: Start
Step 2: Define the class Person:
- Data members: name, gender, address, age
- Constructor to initialize these data members

Step 3: Define the class Employee that extends Person:
- Additional data members: empId, companyName, qualification, salary
- Constructor to initialize both Person and Employee data members using super() for inheritance

Step 4: Define the class Teacher that extends Employee:
- Additional data members: subject, department, teacherId
- Constructor to initialize both Person, Employee, and Teacher data members using super()
- Method display() to print details of the teacher, including all inherited attributes from Person and Employee

Step 5: In the main() method (in class co3_3):
- Create an array teachers of size 2 to store Teacher objects
- Create two Teacher objects with appropriate data and assign to teachers[0] and teachers[1]
- Call display() method on both teacher objects to print the details

Step 6: End

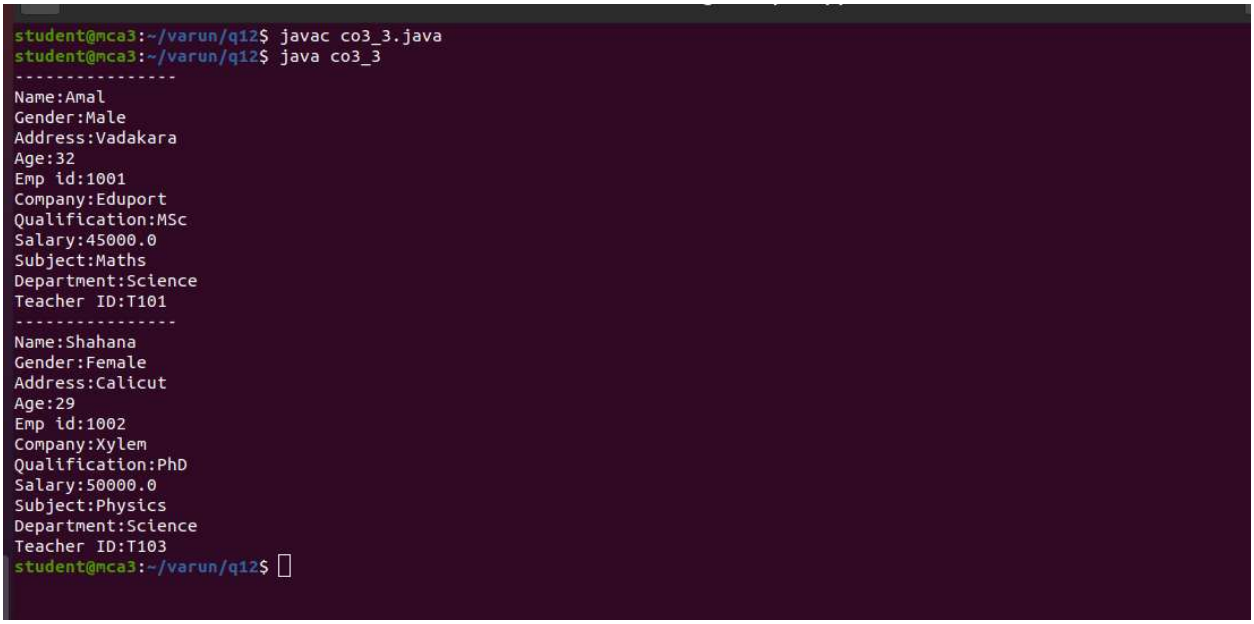
SOURCE CODE:

```
class Person{
String name;
String gender;
String address;
int age;

Person(String name,String gender,String address,int age){
this.name=name;
this.gender=gender;
this.address=address;
```

```
this.age=age;
}
}

class Employee extends Person{
int empid;
String companyName;
String qualification;
double salary;
Employee(String name,String gender,String address,int age,int empid,String companyName,String
qualification,double salary){
super(name,gender,address,age);
this.empid=empid;
this.companyName=companyName;
this.qualification=qualification;
this.salary=salary;
}
}

class Teacher extends Employee{
String subject;
String department;
String teacherId;
Teacher(String name,String gender,String address,int age,int empid,String companyName,String
qualification,double salary,String subject,String department,String teacherId){
super(name,gender,address,age,empid,companyName,qualification,salary);
this.subject=subject;
this.department=department;
this.teacherId=teacherId;
}

void display(){
System.out.println("----------------");
System.out.println("Name:"+name);
System.out.println("Gender:"+gender);
System.out.println("Address:"+address);
System.out.println("Age:"+age);
System.out.println("Emp id:"+empid);
System.out.println("Company:"+companyName);
System.out.println("Qualification:"+qualification);
System.out.println("Salary:"+salary);
System.out.println("Subject:"+subject);
System.out.println("Department:"+department);
System.out.println("Teacher ID:"+teacherId);
}
}
```

```
public class co3_3{
public static void main(String[]args){
Teacher[] teachers=new Teacher[2];
teachers[0]=new
Teacher("Amal","Male","Vadakara",32,1001,"Eduport","MSc",45000,"Maths","Science","T101");
teachers[1]=new
Teacher("Shahana","Female","Calicut",29,1002,"Xylem","PhD",50000,"Physics","Science","T10
3");
teachers[0].display();
teachers[1].display();
}
}
```

OUTPUT:



```
student@mca3:~/varun/q12$ javac co3_3.java
student@mca3:~/varun/q12$ java co3_3
----------------
Name:Amal
Gender:Male
Address:Vadakara
Age:32
Emp id:1001
Company:Eduport
Qualification:MSc
Salary:45000.0
Subject:Maths
Department:Science
Teacher ID:T101
----------------
Name:Shahana
Gender:Female
Address:Calicut
Age:29
Emp id:1002
Company:Xylem
Qualification:PhD
Salary:50000.0
Subject:Physics
Department:Science
Teacher ID:T103
student@mca3:~/varun/q12$
```

RESULT:

Successfully executed

PROGRAM 13:

AIM:

Write a program has class Publisher, Book, Literature and Fiction. Read the information and print the details of books from either the category, using inheritance.

ALGORITHM:

Step 1: Start
Step 2: Define the Publisher class:
- Data member: publisherName
- Constructor to initialize publisherName

Step 3: Define the Book class that extends Publisher:
- Additional data members: title, author
- Constructor to initialize publisherName from Publisher class and title, author for Book class

Step 4: Define the Literature class that extends Book:
- Constructor to initialize publisherName, title, and author by calling the super() constructor
- Method display() to print the details specific to the Literature category (Category, Title, Author, Publisher)

Step 5: Define the Fiction class that extends Book:
- Constructor to initialize publisherName, title, and author by calling the super() constructor
- Method display() to print the details specific to the Fiction category (Category, Title, Author, Publisher)

Step 6: In the main() method (in class BookStore):
- Prompt user to enter the number of Literature books, read the number, and store them in literatureBooks array
- For each Literature book, prompt user to enter the title, author, and publisher, then create a Literature object and store it in literatureBooks
- Prompt user to enter the number of Fiction books, read the number, and store them in fictionBooks array
- For each Fiction book, prompt user to enter the title, author, and publisher, then create a Fiction object and store it in fictionBooks

Step 7: Display details of all Literature books:
- Loop through literatureBooks and call display() method for each book

Step 8: Display details of all Fiction books:
- Loop through fictionBooks and call display() method for each book

Step 9: End

SOURCE CODE:

```
import java.util.Scanner;
class Publisher
{
String publisherName;
```

```
Publisher(String publisherName)
{
this.publisherName=publisherName;
}
}
class Book extends Publisher
{
String title;
String author;
Book(String publisherName,String title,String author)
{
super(publisherName);
this.title=title;
this.author=author;
}
}
class Literature extends Book
{
Literature(String publisherName,String title,String author)
{
super(publisherName,title,author);
}
void display()
{
System.out.println("Category: Literature");
System.out.println("Title:"+title);
System.out.println("Author:"+author);
System.out.println("Publisher:"+publisherName);
System.out.println("-----------------------");
}
}
class Fiction extends Book
{
Fiction(String publisherName,String title,String author)
{
super(publisherName,title,author);
}
void display()
{
System.out.println("Category: Fiction");
System.out.println("Title:"+title);
System.out.println("Author:"+author);
System.out.println("Publisher:"+publisherName);
System.out.println("-----------------------");
}
}
```

```
public class BookStore
{
public static void main(String[]args)
{
Scanner sc=new Scanner(System.in);
System.out.println("Enter number of literature books:");
int n=sc.nextInt();
sc.nextLine();
Literature[] literatureBooks=new Literature[n];
for(int i=0;i<n;i++)
{
System.out.println("Enter details for literature book "+(i+1));
System.out.println("Title:");
String title=sc.nextLine();
System.out.println("Author:");
String author=sc.nextLine();
System.out.println("Publisher:");
String publisher=sc.nextLine();
literatureBooks[i]=new Literature(publisher,title,author);
}
System.out.println("Enter number of fiction books:");
int n1=sc.nextInt();
sc.nextLine();
Fiction[] fictionBooks=new Fiction[n1];
for(int i=0;i<n1;i++)
{
System.out.println("Enter details for fiction book "+(i+1));
System.out.println("Title:");
String title=sc.nextLine();
System.out.println("Author:");
String author=sc.nextLine();
System.out.println("Publisher:");
String publisher=sc.nextLine();
fictionBooks[i]=new Fiction(publisher,title,author);
}
System.out.println("\n-----------Literature Books-----------");
for(Literature book:literatureBooks)
{
book.display();
}
System.out.println("\n-----------Fiction Books-----------");
for(Fiction book:fictionBooks)
{
book.display();
}
sc.close();
```

}
}

OUTPUT:

```
student@mca3:~/varun/q13$ java BookStore.java
Error: Could not find or load main class BookStore.java
student@mca3:~/varun/q13$ java BookStore
Enter number of literature books:
1
Enter details for literature book 1
Title:
Hamlet
Author:
William shakespeare
Publisher:
DC
Enter number of fiction books:
1
Enter details for fiction book 1
Title:
Harry potter
Author:
j k Rowling
Publisher:
DC

-----------Literature Books-----------
Category: Literature
Title:Hamlet
Author:William shakespeare
Publisher:DC
----------------------

-----------Fiction Books-----------
Category: Fiction
Title:Harry potter
Author:j k Rowling
Publisher:DC
----------------------
student@mca3:~/varun/q13$
```

RESULT:

Successfully executed

PROGRAM 14:

AIM:

Create classes Student and Sports. Create another class Result inherited from Student and Sports. Display the academic and sports score of a student.

ALGORITHM:

Step 1: Start
Step 2: Define an Interface Sports
- Declare two abstract methods:
  sports_getData()
  sports_dispData()

Step 3: Define the Student class
- Data Members: name, roll_no, score
- Constructor:
  - Accepts name, roll_no, and score (academic score)
  - Initializes the data members
- Method display():
  - Displays the student's name, roll number, and academic score

Step 4: Define the Result class that extends Student and implements Sports interface
- Data Member: score (sports score)
- Constructor:
  - Calls the superclass constructor to initialize student data
- Method sports_getData():
  - Prompts the user to enter the sports score
  - Reads and stores it
- Method sports_dispData():
  - Displays the sports score

Step 5: In the main() method (class co3_5):
- Create a Scanner object
- Read student name, roll number, and academic score from the user
- Create an object of Result class with the entered data
- Call sports_getData() to input sports score
- Call display() to show academic details
- Call sports_dispData() to show sports score

Step 6: End

SOURCE CODE:

```
import java.util.*;
import java.io.*;
interface Sports
{
public void sports_getData();
public void sports_dispData();
}
class Student
```

```
{
String name;
int roll_no;
int score;
Student(String name,int roll,int m1)
{
this.name=name;
this.roll_no=roll;
this.score=m1;
}
void display()
{
System.out.println("Student details");
System.out.println("Name of student:"+name);
System.out.println("Roll no:"+roll_no);
System.out.println("Academic score:"+score);
}
}
class Result extends Student implements Sports
{
int sportsScore;
Scanner sc=new Scanner(System.in);
Result(String name,int roll,int m1)
{
super(name,roll,m1);
}
public void sports_getData()
{
System.out.println("enter the score you have obtain in sports");
sportsScore=sc.nextInt();
}
public void sports_dispData()
{
System.out.println("sports score:"+sportsScore);
}
}
class co3_5
{
public static void main(String args[])
{
String name;
int roll,mark;
Scanner sc=new Scanner(System.in);
System.out.println("enter the name");
name=sc.nextLine();
System.out.println("enter the roll");
```

```
roll=sc.nextInt();
System.out.println("enter the academic score:");
mark=sc.nextInt();
Result re=new Result(name,roll,mark);
re.sports_getData();
re.display();
re.sports_dispData();
}
}
```

OUTPUT:

```
student@mca3:~/varun/q14$ java co3_5
enter the name
vrun
enter the roll
45
enter the academic score:
100
enter the score you have obtain in sports
60
Student details
Name of student:vrun
Roll no:45
Academic score:100
sports score:60
student@mca3:~/varun/q14$
```

RESULT:

Successfully executed

PROGRAM 15:

AIM:

Create an interface having prototypes of functions area() and perimeter(). Create two classes Circle and Rectangle which implements the above interface. Create a menu driven program to find area and perimeter of objects.

ALGORITHM:

Step 1: Start
Step 2: Define an Interface Shape
  2.1 Declare two methods:
    - double area()
    - double perimeter()
Step 3: Define Class Circle Implementing Shape
  3.1 Declare radius as a data member
  3.2 Define constructor to initialize radius
  3.3 Implement:
    - area() as π * radius * radius
    - perimeter() as 2 * π * radius
Step 4: Define Class Rectangle Implementing Shape
  4.1 Declare length and width as data members
  4.2 Define constructor to initialize length and width
  4.3 Implement:
    - area() as length * width
    - perimeter() as 2 * (length + width)
Step 5: In the main() Method
  5.1 Create Scanner object for input
  5.2 Declare choice variable
  5.3 Repeat indefinitely using a while(true) loop:
    - Display menu options:
      - Circle
      - Rectangle
      - Exit
    - Prompt user to enter choice
  5.4 Use switch-case to handle choice:
    - Case 1:
      - Ask user to enter radius
      - Create a Circle object
      - Display:
        circle.area()
        circle.perimeter()
    - Case 2:
      - Ask user to enter length and width
      - Create a Rectangle object
      - Display rectangle.area(),rectangle.perimeter()

- Case 3:
    - Print exit message
    - Exit the loop using return
- Default:
    - Print "Invalid choice" message

Step 6: End

SOURCE CODE:

```
import java.util.Scanner;
interface Shape{
double area();
double perimeter();
}
class Circle implements Shape{
double radius;
Circle(double radius){
this.radius=radius;
}
public double area(){
return Math.PI*radius*radius;
}
public double perimeter(){
return 2*Math.PI*radius;
}
}
class Rectangle implements Shape{
double length;
double width;
Rectangle(double length,double width){
this.length=length;
this.width=width;
}
public double area(){
return length*width;
}
public double perimeter(){
return 2*(length+width);
}}
public class co3_6{
public static void main(String[]args){
Scanner sc=new Scanner(System.in);
int choice;
while(true){
System.out.println("\nMenu");
System.out.println("1.Calculate area and perimeter of the circle");
System.out.println("2.Calculate area and perimeter of rectangle");
System.out.println("3.Exit");
```

```
System.out.println("Enter your choice:");
choice=sc.nextInt();
switch(choice){
case 1:
System.out.print("Enter the radius:");
double radius=sc.nextDouble();
Circle c=new Circle(radius);
System.out.println("Area of circle:"+c.area());
System.out.println("Perimeter of circle:"+c.perimeter());
break;
case 2:
System.out.print("Enter length of the rectangle:");
double length=sc.nextDouble();
System.out.print("Enter width of the rectangle:");
double width=sc.nextDouble();
Rectangle r=new Rectangle(length,width);
System.out.println("Area of rectangle:"+r.area());
System.out.println("Perimeter of rectangle:"+r.perimeter());
break;
case 3:
System.out.println("Exiting program");
return;
default:
System.out.println("Invalid choice please try again");}}}}
```

OUTPUT:

```
student@mca3:~/varun/q15$ java co3_6

Menu
1.Calculate area and perimeter of the circle
2.Calculate area and perimeter of rectangle
3.Exit
Enter your choice:
1
Enter the radius:5
Area of circle:78.53981633974483
Perimeter of circle:31.41592653589793

Menu
1.Calculate area and perimeter of the circle
2.Calculate area and perimeter of rectangle
3.Exit
Enter your choice:
2
Enter length of the rectangle:3
Enter width of the rectangle:4
Area of rectangle:12.0
Perimeter of rectangle:14.0

Menu
1.Calculate area and perimeter of the circle
2.Calculate area and perimeter of rectangle
3.Exit
Enter your choice:
3
Exiting program
student@mca3:~/varun/q15$
```

RESULT:

Successfully executed

PROGRAM 16:

AIM:

Prepare bill with the given format using calculate method from interface. Order No.

ALGORITHM:

Step 1: Start
Step 2: Define an Interface Bill
  2.1 Declare a method: void calculate()
Step 3: Define a Class Product that Implements Bill
  3.1 Declare data members:
    - int productId, int quantity
    - String name
    - float unitPrice, float total
  3.2 Define method getData() to:
    - Take input for all product details from the user
  3.3 Define method calculate() to:
    - Compute total = quantity * unitPrice
  3.4 Define method display() to:
    - Print product details
  3.5 Define method getTotal() to:
    - Return the value of total
Step 4: In the main() Method
  4.1 Create Scanner object for user input
  4.2 Prompt user to enter:
    - Order number → orderNo
    - Number of products → n
  4.3 Create an array of Product objects of size n
    - Initialize netAmount = 0
Step 5: For Each Product
  5.1 Create a new Product object
    - Call:
      - getData() to read product info
      - calculate() to compute total price
  5.2 Add each product's total to netAmount
Step 6: Print the Bill
  - Print:
    - Order number and current date
    - Table header for product info
    - Product details by calling display() on each product
    - Print the net amount using formatted output
Step 7: End

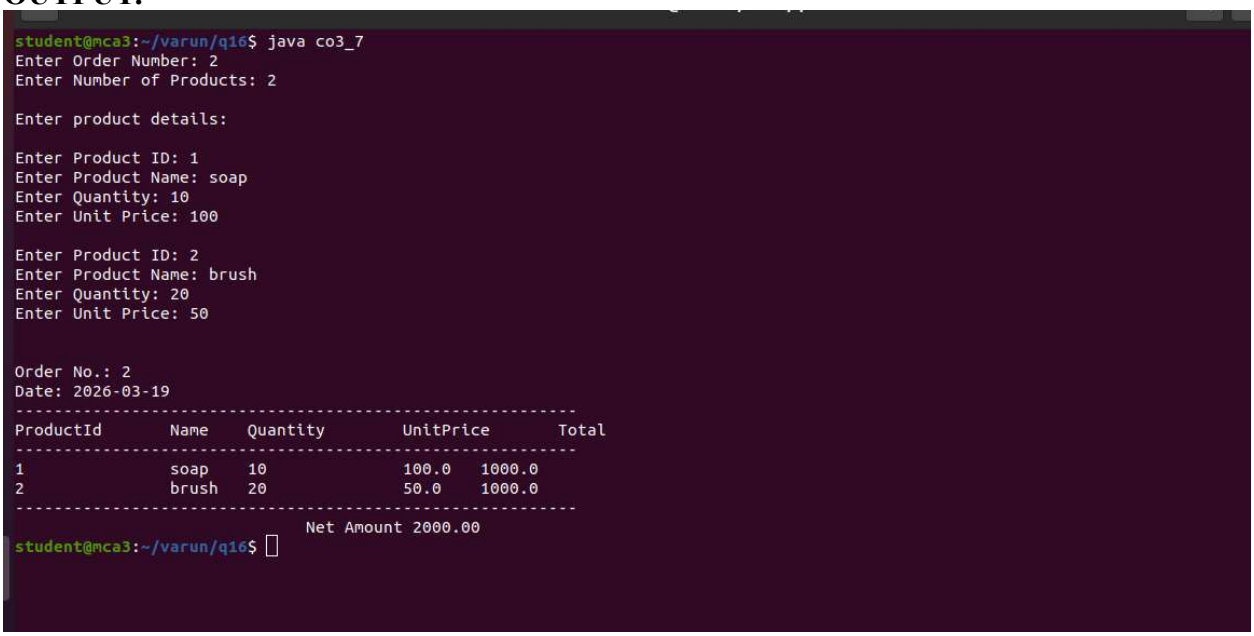
SOURCE CODE:

```
import java.util.*;
interface Bill{
```

```
void calculate();
}
class Product implements Bill{
int productId;
String name;
int quantity;
float unitPrice;
float total;
void getData(){
Scanner sc=new Scanner(System.in);
System.out.print("Enter Product ID: ");
productId=sc.nextInt();
sc.nextLine();
System.out.print("Enter Product Name: ");
name=sc.nextLine();
System.out.print("Enter Quantity: ");
quantity=sc.nextInt();
System.out.print("Enter Unit Price: ");
unitPrice=sc.nextFloat();
}
public void calculate(){
total=quantity*unitPrice;
}
void display(){
System.out.println(productId+"\t\t"+name+"\t"+quantity+"\t\t"+unitPrice+"\t"+total);
}
float getTotal(){
return total;
}
}
public class co3_7{
public static void main(String[] args){
Scanner sc=new Scanner(System.in);
System.out.print("Enter Order Number: ");
int orderNo=sc.nextInt();
System.out.print("Enter Number of Products: ");
int n=sc.nextInt();
Product[] products=new Product[n];
float netAmount=0;
System.out.println("\nEnter product details:\n");
```

```
for(int i=0;i<n;i++){
products[i]=new Product();
products[i].getData();
products[i].calculate();
netAmount+=products[i].getTotal();
System.out.println();
}
System.out.println("\nOrder No.: "+orderNo);
System.out.println("Date: "+java.time.LocalDate.now());
System.out.println("-------------------------------------------------------");
System.out.println("ProductId\tName\tQuantity\tUnitPrice\tTotal");
System.out.println("-------------------------------------------------------");
for(Product p:products){
p.display();
}
System.out.println("-------------------------------------------------------");
System.out.printf("%40s %-10.2f\n","Net Amount",netAmount);
}
}
```

OUTPUT:



```
student@mca3:~/varun/q16$ java co3_7
Enter Order Number: 2
Enter Number of Products: 2

Enter product details:

Enter Product ID: 1
Enter Product Name: soap
Enter Quantity: 10
Enter Unit Price: 100

Enter Product ID: 2
Enter Product Name: brush
Enter Quantity: 20
Enter Unit Price: 50


Order No.: 2
Date: 2026-03-19
-------------------------------------------------------
ProductId       Name    Quantity        UnitPrice       Total
-------------------------------------------------------
1               soap    10              100.0   1000.0
2               brush   20              50.0    1000.0
-------------------------------------------------------
                                Net Amount 2000.00
student@mca3:~/varun/q16$
```

RESULT:

Successfully executed

PROGRAM 17:

AIM:

Create a Graphics package that has classes and interfaces for figures Rectangle, Triangle, Square and Circle. Test the package by finding the area of these figures.

ALGORITHM:

Step 1: Start
Step 2: Define a Package Graphics
Step 3: Inside the Package, Define an Interface inter_graph
  3.1 Declare abstract methods:
    - int rect(int l, int w) → for area of rectangle
    - double tria(double b, double h) → for area of triangle
    - float squ(int a) → for area of square
    - double cir(double r) → for area of circle
Step 4: Define a Class area that Implements inter_graph
  4.1 Implement each method:
    - rect(l, w) → return l * w
    - tria(b, h) → return 0.5 * b * h
    - squ(a) → return a * a
    - cir(r) → return π * r * r
Step 5: Create Another File TestGra.java Outside the Package
  5.1 Import Graphics.area class
  - Define class TestGra with main() method
Step 6: In main()
  6.1 Create object g of class area
  6.2 Call methods using the object:
    - g.rect(4,5) and print result
    - g.tria(3,4.5) and print result
    - g.squ(5) and print result
    - g.cir(4.5) and print result
Step 7: End

SOURCE CODE:

Area.java

```
interface inter_graph
{
public int rect(int l,int w);
public double tria(double b,double h);
public float squ(int a);
public double cir(double r);
}
public class area implements inter_graph
```

```
{
public int rect(int l,int w)
{
return l*w;
}
public double tria(double b,double h)
{
return 0.5*b*h;
}
public float squ(int a)
{
return a*a;
}
public double cir(double r)
{
return Math.PI*r*r;
}
}
```

Testgra.java

```
public class TestGra
{
public static void main(String[]args)
{
area g=new area();
System.out.println("area of rectangle:"+g.rect(4,5));
System.out.println("area of triangle:"+g.tria(3,4.5));
System.out.println("area of square:"+g.squ(5));
System.out.println("area of circle:"+g.cir(4.5));
}
}
```

OUTPUT:

```
student@mca3:~/varun/q17$ java TestGra
area of rectangle:20
area of triangle:6.75
area of square:25.0
area of circle:63.61725123519331
student@mca3:~/varun/q17$
```

RESULT:

Successfully executed

PROGRAM 18:

AIM:

Create an Arithmetic package that has classes and interfaces for the 4 basic arithmetic operations. Test the package by implementing all operations on two given numbers.

ALGORITHM:

Step 1: Start
Step 2: Define a Package Arithmetic
Step 3: Inside the Package, Define an Interface inter_arith
  3.1 Declare the following abstract methods:
    - float add(int a, int b)
    - float sub(int a, int b)
    - float mult(int a, int b)
    - float div(int a, int b)
    - float rem(int a, int b)
Step 4: Define a Class oprtn that Implements inter_arith
  4.1 Implement all the methods:
    - add(a, b) → return a + b
    - sub(a, b) → return a - b
    - mult(a, b) → return a * b
    - div(a, b) → return a / b
    - rem(a, b) → return a % b
Step 5: Create Another File TestArit.java Outside the Package
  5.1 Import the Arithmetic package
  5.2 Define class TestArit with the main() method
Step 6: In the main() Method
  6.1 Create object obj of class oprtn
  6.2 Call and display the results of:
    - obj.add(45, 4) → Print result
    - obj.sub(34, 18) → Print result
    - obj.mult(9, 4) → Print result
    - obj.div(45, 4) → Print result
    - obj.rem(45, 10) → Print result
Step 7: End

SOURCE CODE:

Oprtn.java

```
interface inter_arith
{
float add(int a,int b);
```

```
float sub(int a,int b);
float mult(int a,int b);
float div(int a,int b);
float rem(int a,int b);
}
public class oprtn implements inter_arith
{
public float add(int a,int b)
{return a+b;}
public float sub(int a,int b)
{return a-b;}
public float mult(int a,int b)
{return a*b;}
public float div(int a,int b)
{return (float)a/b;}
public float rem(int a,int b)
{return a%b;}
}
```

Testarith.java

```
public class TestArith
{
   public static void main (String[]args)
   {
      oprtn obj=new oprtn();
      System.out.println("addition:"+obj.add(45,5));
      System.out.println("substraction:"+obj.sub(34,18));
      System.out.println("multipication:"+obj.mult(9,4));
      System.out.println("division:"+obj.div(45,4));
      System.out.println("reminder:"+obj.rem(45,10));
   }
}
```

OUTPUT:

```
student@mca3:~/varun/q18$ java TestArith
addition:50.0
substraction:16.0
multipication:36.0
division:11.25
reminder:5.0
student@mca3:~/varun/q18$
```

RESULT:

Successfully executed